DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Artificial Intelligence & Machine Learning)

Academic Year:	2026-27	**Name of Student:**	
Semester:	V	**Student ID:**	
Class / Branch:	TE – CSE (AI&ML)	**Roll No:**	
Subject:	WMD Lab	**Date of Submission:**	

Experiment No. 03

Lab Outcome: Develop responsive front-end applications using React components and state management.

Aim: Create a simple React app with components and JSX; display static product/user data.

Software used: JavaScript, NodeJS, React.

Theory: -

1. Introduction to React

React is an open-source JavaScript library used for building user interfaces, particularly for web applications. It was developed by Facebook and is now maintained by Meta and the open-source community.

React follows a component-based architecture, in which a large user interface is divided into smaller, independent, and reusable components. Each component is responsible for a specific part of the user interface. For example, a product catalogue application can be divided as follows:

```
App
 │
 ├── Header
 ├── FilterBar
 └── ProductGrid
      └── ProductCard
```

This approach makes applications easier to develop, understand, maintain, and reuse.

2. React Components and Functional Components

A component is an independent and reusable building block of a React application. In modern React applications, components are generally created using JavaScript functions, called functional components — a plain JavaScript function that returns JSX.

Example (function declaration syntax):

```
function Header() {
  return (
    <header>
      <h1>TechStore</h1>
      <p>Engineering Lead</p>
    </header>
  );
}

export default Header;
```

The same component can also be written using arrow-function syntax, which is common in modern React code:

```
const Header = () => {
  return (
    <header>
      <h1>TechStore</h1>
    </header>
  );
};
```

The Header component returns JSX that describes the user interface to be displayed. A component can be reused wherever it is required, and functional components are preferred in modern React because they are simple, reusable, and work naturally with React Hooks.

3. JSX and JSX Rules

JSX (JavaScript XML) is a syntax extension for JavaScript used in React to describe the structure of the user interface. It allows developers to write HTML-like syntax directly inside JavaScript code, and to embed JavaScript expressions inside curly braces { }:

```
const productName = "Wireless Headphones";

return (
  <div>
    <h2>{productName}</h2>
    <p>Premium noise-cancelling headphones</p>
  </div>
);
```

JSX makes React code easier to read because the structure of the user interface and the logic used to generate it can be written together. Some important rules of JSX are:

Rule 1 — JSX must have a single parent element

```
return (
```

```
  <div>
    <h1>Product</h1>
    <p>Wireless Headphones</p>
  </div>
);
```

Alternatively, a React Fragment (<>...</>) can be used to group elements without adding an extra DOM node:

```
return (
  <>
    <h1>Product</h1>
    <p>Wireless Headphones</p>
  </>
);
```

Rule 2 — Use className instead of class

HTML uses class, but since class is a reserved word in JavaScript, JSX uses className instead:

```
HTML:  <div class="product-card">
JSX:   <div className="product-card">
```

Rule 3 — JavaScript expressions are written inside { }

```
<p>Price: £{price}</p>
```

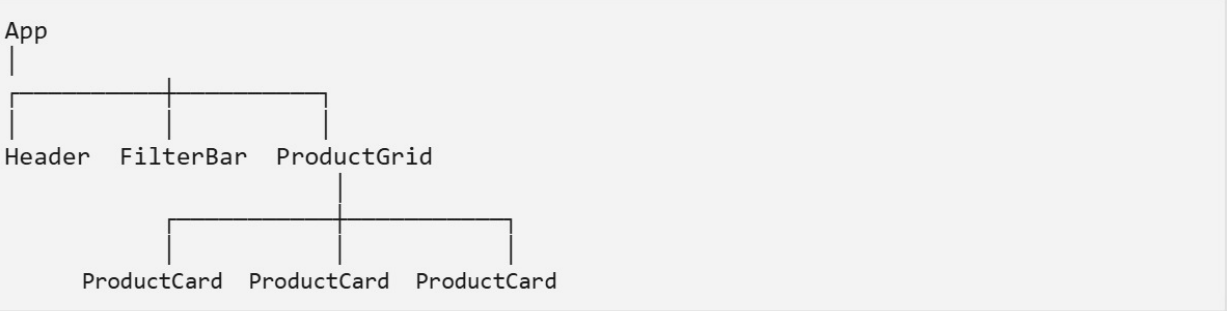
Rule 4 — JSX attributes generally use camelCase

For example, the HTML onclick attribute becomes onClick in JSX:

```
<button onClick={handleClick}>
  Add to Cart
</button>
```

4. Component-Based Architecture

The product catalogue application used in this experiment follows a component-based structure, both conceptually and as a component hierarchy:



This architecture demonstrates component composition, where multiple smaller components are combined to build a complete application. For example, the App component combines the major UI sections:

```
function App() {
  return (
    <>
      <Header />
      <h1>Product Catalog</h1>
      <FilterBar />
      <ProductGrid />
    </>
  );
}
```

This makes the application modular and easy to understand. Each file in the project has a specific responsibility:

File	Responsibility
App.jsx	Acts as the main application component and combines the other components.
Header.jsx	Displays static user information such as user name, user role, and application branding.
FilterBar.jsx	Displays the product search field, category selection, and stock filter.
ProductGrid.jsx	Receives the product data and creates a collection of product cards.
ProductCard.jsx	Displays information about an individual product.
mockData.js	Stores static product and user data separately from the UI components.

Separating data from UI components in a dedicated file (such as mockData.js) improves code organisation and makes the data easier to modify.

5. Static Data in React

In this experiment, the application uses static data, meaning the data is predefined within the application rather than retrieved from an external database or API.

Product Data

The product catalogue holds information such as product name, category, price, rating, availability, and image, represented as JavaScript objects, for example:

```
const products = [
  {
    id: 1,
    name: "Wireless Noise-Cancelling Headphones",
    category: "Electronics",
```

```
    price: 299.99,
    rating: 4.8,
    inStock: true
  },
  {
    id: 2,
    name: "Mechanical RGB Gaming Keyboard",
    category: "Accessories",
    price: 129.50,
    rating: 4.6,
    inStock: true
  }
];
```

User Data

Similarly, the header can display static user information, represented as a single JavaScript object:

```
const user = {
  name: "Vijesh Nair",
  role: "Engineering Lead"
};
```

It can then be displayed using JSX:

```
<h2>{user.name}</h2>
<p>{user.role}</p>
```

The data is typically stored in a separate file, such as src/data/mockData.js, and imported into the component that needs it:

```
import { products } from "../data/mockData";
```

Separating data from UI components is a good programming practice because it improves readability and maintainability.

6. Props in React

Props, short for properties, are used to pass data from a parent component to a child component. For example, ProductGrid can pass individual product information to ProductCard:

```
<ProductCard product={product} />
```

The child component then receives the prop as a function parameter:

```
function ProductCard({ product }) {
  return (
    <div>
      <h2>{product.name}</h2>
      <p>Price: £{product.price}</p>
```

```
    </div>
  );
}
```

The data flow can therefore be represented as:

```
App
  | products
  ▼
ProductGrid
  | product
  ▼
ProductCard
```

React follows a one-way data flow, where data generally moves from parent components to child components through props.

7. Rendering Multiple Products with map()

React applications often need to display collections of data. The JavaScript map() method can be used to dynamically generate JSX elements:

```
function ProductGrid({ products }) {
  return (
    <div className="product-grid">
      {products.map((product) => (
        <ProductCard
          key={product.id}
          product={product}
        />
      ))}
    </div>
  );
}
```

Here, map() iterates through the products array, a ProductCard is created for each product, product is passed to it as a prop, and key uniquely identifies each rendered item. The key attribute is important when rendering lists in React because it helps React efficiently identify which items have changed.

8. Conditional Rendering

React supports conditional rendering, which allows different UI elements to be displayed based on a condition. For example, the application can display different buttons depending on whether a product is available:

```
{product.inStock ? (
  <button>Add to Cart</button>
) : (
```

```
  <button disabled>Out of Stock</button>
)}
```

If inStock is true, the user sees "Add to Cart"; if inStock is false, the user sees "Out of Stock" instead. This demonstrates how React can dynamically control the user interface based on data.

9. CSS Styling in React

React separates the structure of the interface from its presentation — CSS is used to style React components in the usual way, by class name:

```
<div className="product-card">
```

with a corresponding CSS rule such as:

```
.product-card {
  background: white;
  border-radius: 8px;
  padding: 20px;
}
```

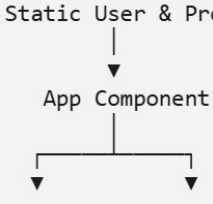
In the product catalogue application, CSS is used to style the header layout, product grid, product cards, search and filter controls, buttons, product images, ratings, and stock status. The combination of React components, JSX, and CSS produces the final user interface shown in the output.

10. Advantages of Using Components and JSX

- Reusability — components can be reused in different parts of an application.
- Maintainability — smaller components are easier to modify and debug.
- Readability — the application structure is easier to understand.
- Separation of Concerns — different UI responsibilities can be handled by different components.
- Scalability — large applications can be built by combining smaller components.
- Dynamic UI — JSX allows JavaScript data to be dynamically rendered.
- Efficient Development — developers can work on individual components independently.

11. Expected Conceptual Flow

The overall working of the application can be represented as:





Thus, the experiment demonstrates how static data is stored, passed through components using props, processed using JavaScript, and rendered using JSX. A simple React-based Product Catalog application is developed using functional components and JSX, dividing the interface into reusable components such as Header, FilterBar, ProductGrid, and ProductCard. Static user and product information is maintained separately in a data file and rendered dynamically using JSX and JavaScript.

The experiment provides a foundation for understanding important React concepts such as component-based architecture, JSX, props, list rendering, conditional rendering, static data management, and CSS styling. These concepts form the basis for developing more advanced React applications that can later integrate APIs, state management, routing, and backend services.

Implementation

Application Overview

The provided application implements a Product Catalog interface containing:

- A header displaying user information.
- A product catalogue heading.
- A search input, a category selection dropdown, and an "In Stock Only" checkbox.
- Multiple product cards, showing product images, categories, ratings, and prices.
- "Add to Cart" buttons, and an "Out of Stock" status for unavailable products.

The project structure shown in the VS Code Explorer indicates that the application has been divided into reusable components — a good example of separation of concerns in React application development:

```
src
├── components
│   ├── FilterBar.jsx
│   ├── Header.jsx
│   ├── ProductCard.jsx
│   └── ProductGrid.jsx
├── data
│   └── mockData.js
├── App.css
```

```
├── App.jsx
├── index.css
└── main.jsx
```

Role of Important Files

File / Folder	Purpose
main.jsx	Entry point of the React application.
App.jsx	Root application component.
App.css	Styling specific to the application.
index.css	Global CSS styling.
Header.jsx	Displays application header and user information.
FilterBar.jsx	Displays search and filtering controls.
ProductGrid.jsx	Organises and renders multiple product cards.
ProductCard.jsx	Displays individual product details.
mockData.js	Stores static user and product data.
public	Contains publicly accessible static resources.
package.json	Contains project metadata, dependencies, and scripts.
vite.config.js	Configuration file for the Vite development environment.

Installation Procedures

Install Node.js and npm using NVM (Node Version Manager)

Step 1 — Install Node Version Manager (NVM)

Run the following command in the terminal:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
```

Activate NVM using the command below:

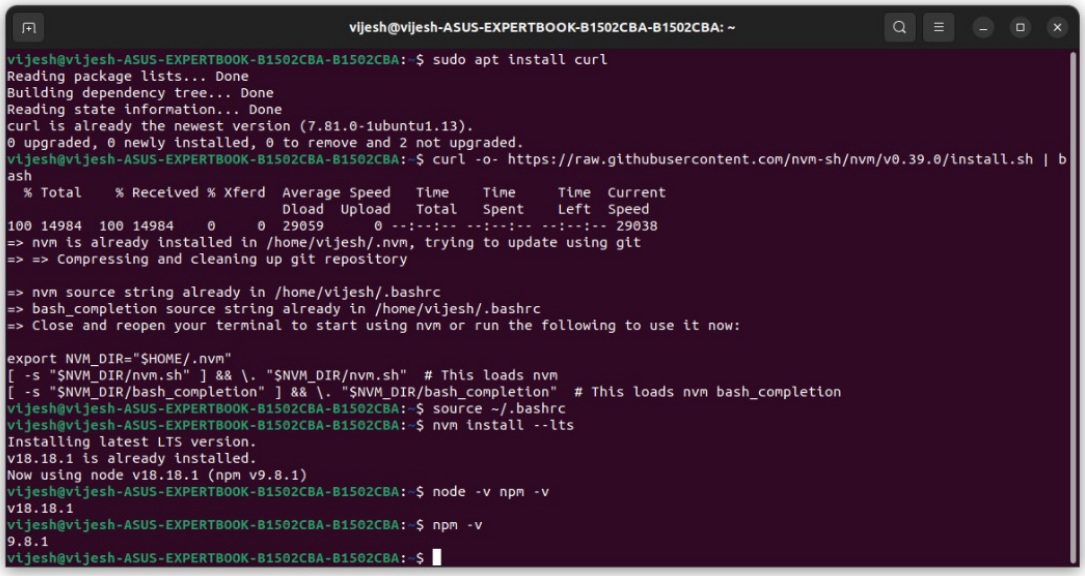
```
source ~/.bashrc
```

Step 2 — Install the latest LTS version of Node.js

```
nvm install --lts
```

Step 3 — Confirm the installation of Node.js

```
node -v
npm -v
```



Installing React on Windows — A Step-by-Step Guide

Installing React on Windows involves three phases: preparing your system, creating the project, and launching it. Follow these steps exactly to build the first React app using Vite, which is the modern standard recommended by the React team.

Phase 1: Preparing Your Computer

Before building a React app, your computer needs a runtime engine called Node.js. This acts like an engine that executes JavaScript code outside of a web browser.

1. Open your web browser and go to the official website: nodejs.org.
2. Click the green button labeled **LTS (Long Term Support)**. This version is safest for students because it is stable.
3. Wait for the installer file (.msi) to finish downloading, then double-click it to open the installation wizard.
4. Click Next, accept the license agreement, keep clicking Next on the default settings, and click Install, then Finish.

Important: when the checkbox asking to install "Tools for Native Modules" (Chocolatey) appears, leave it unchecked — beginners do not need it.

Phase 2: Creating Your First React Project

By default, the Command Prompt opens in the user folder. Move to the Desktop so the project folder is easy

to find:

```
cd Desktop
```

Instead of the older create-react-app, this experiment uses Vite, which is faster and up to date:

```
npm create vite@latest my-first-react-app
```

The system may ask: Need to install the following packages: create-vite? (y/n) — type y and press Enter.

Move the Command Prompt inside the newly created project folder:

```
cd my-first-react-app
```

Download all required background files:

```
npm install
```

Wait a minute or two. A success message appears once installation finishes.

Phase 3: Launching the Project

Start the local development server:

```
npm run dev
```

The terminal displays a local address, typically:

```
➜  Local:   http://localhost:5173/
```

Opening this address in the browser displays a live website with a spinning React logo.

Stopping the App

Return to the Command Prompt window and press Ctrl + C. If prompted Terminate batch job (Y/N)?, type y and press Enter — the window can then be closed safely.

Application Boot Sequence and Execution Flow

The following describes the step-by-step boot sequence and execution flow of the React application — from the moment the browser opens to when a user clicks a button.

Phase 1 — Application Boot Sequence (Static Load)

When a modern React application starts up (using build tools such as Vite), execution begins at the HTML entry point and flows downward into the component files:

```
1. public/index.html ⟶ 2. src/main.jsx ⟶ 3. src/App.jsx ⟶ 4. Child Modules & Components
```

public/index.html (the web page shell) — the browser opens this HTML file first. It contains a single container

:

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

src/main.jsx (the React mount point) — this file runs first inside the JavaScript runtime. It finds #root in the HTML and injects React into it:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';   // Loads App.jsx

ReactDOM.createRoot(document.getElementById('root')).render(<App />);
```

Phase 2 — Import Order (src/App.jsx)

When App.jsx loads, JavaScript evaluates all its top-level import statements before running the App() component function:

```
src/App.jsx
│
├── 1. Imports CSS Styles     ──► src/App.css
├── 2. Imports Static Data    ──► src/data/mockData.js
├── 3. Imports Top Header     ──► src/components/Header.jsx
├── 4. Imports Filter Controls──► src/components/FilterBar.jsx
└── 5. Imports Grid Container ──► src/components/ProductGrid.jsx
                                    └── Imports ──► src/components/ProductCard.jsx
```

Phase 3 — Initial Render Flow (Top-Down Execution)

Once the imported files are ready, React runs the code top-to-bottom inside App.jsx:

```
[ App.jsx Execution ]
        │
1. Initialize State (useState)
        │
2. Calculate Derived State (filter())
        │
3. Evaluate Return JSX (Render Children)
        │
┌───────┼────────┐
▼       ▼        ▼
Header FilterBar ProductGrid
                    │
              ProductCard (Loop x4)
```

1. State Initialization (*App.jsx*): sets products to INITIAL_PRODUCTS, cart to [], and searchQuery to "".
2. Derived Calculations (*App.jsx*): computes categories dynamically and runs the filteredProducts logic.

3. JSX Rendering (top to bottom): Header.jsx receives CURRENT_USER, totalCartItems, and totalCartCost as props to draw the top bar; FilterBar.jsx receives filter states and setter functions to render its controls; ProductGrid.jsx receives filteredProducts; ProductGrid then maps through the list and invokes ProductCard four separate times, once for each item in mockData.js.

Phase 4 — Runtime User Interaction Flow (Bottom-Up State Update)

React uses unidirectional data flow: data flows down through props, while user actions trigger state updates that flow back up to the parent. For example, clicking "Add to Cart":

```
User Action (Click "Add to Cart")
        │
        ▼
1. [ProductCard.jsx] Calls onAddToCart(product) prop
        │
        ▼
2. [App.jsx] Triggers handleAddToCart() in the parent
        │
        ▼
3. [App.jsx] Updates state via setCart(...)
        │
        ▼
4. [App.jsx] React detects the state change ──► RE-RENDERS APP
        │
        ├──► Header.jsx updates the cart badge item count
        └──► ProductCard.jsx updates its button label with the new quantity
```

Summary of Component Execution Roles

File	Execution Role	Communication Type
main.jsx	App Launcher	Injects React into the browser DOM.
App.jsx	Brain / Orchestrator	Manages state and performs logic.
mockData.js	Data Supplier	Exports static JSON objects.
Header.jsx	Presenter	Reads props → renders UI.
FilterBar.jsx	User Input Receiver	Emits state updates back to App.jsx.
ProductGrid.jsx	List Iterator	Loops over the product array.
ProductCard.jsx	Leaf Presenter	Displays a single item, emits click events.

Design Principles Applied

- **Lifting State Up:** state that needs to be shared between multiple non-parent components (such as the cart, shared between Header and ProductCard) resides in the nearest common parent, App.jsx.
- **Derived State:** filteredProducts, totalCartItems, and totalCartCost are calculated on-the-fly during rendering. Storing these in a separate state variable instead is a common anti-pattern that leads to synchronization bugs.
- **Single Responsibility Principle (SRP):** Header.jsx only handles user display and cart stats, FilterBar.jsx only controls filter states, and ProductCard.jsx only handles the presentation logic for one card.
- **Immutability in State Updates:** handleAddToCart maps/copies arrays rather than modifying state objects directly.

Source Code

Insert the complete code for each file below. The Theory section illustrates the pattern each file follows (functional components, props, static data, list rendering, and conditional rendering); paste the full, final version of each file here.

1. Data Layer (src/data/mockData.js)

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-3 > react-user-product-catalog > src > data > JS mockData.js > ...

```
// Exporting the current user object for app-wide access
export const CURRENT_USER = {
  // Unique identification string for the user account
  id: "u_9021",
  // Full display name of the user
  name: "Vijesh Nair",
  // Designated job role or access level of the user
  role: "Engineering Lead",
  // Direct CDN URL for rendering user avatar picture
  avatar: "https://lh3.googleusercontent.com/d/1btQ7-oYjkt2J6ipanJ_sjbJY7gqabAxr"
// Closing boundary for user data object
};

// Exporting initial catalog dataset containing array of product objects
export const INITIAL_PRODUCTS = [
  // Beginning definition of first item (p-101)
  {
    // Unique identifier assigned to first product
    id: "p-101",
    // Name title shown on the item card
    title: "Wireless Noise-Canceling Headphones",
    // Unit price in USD for calculation and display
    price: 299.99,
    // Category used for dropdown filtering
    category: "Electronics",
    // Boolean indicator for product stock availability
    inStock: true,
    // Numerical user feedback rating out of 5.0
    rating: 4.8,
    // External image CDN URL for rendering card media
    imageUrl: "https://images.unsplash.com/photo-1505740420928-5e560c06d30e?w=400&auto=format&fit=crop&q=80"
  // Closing boundary of first item definition
  },
```

```
    // Beginning definition of second item (p-102)
    {
      // Unique identifier assigned to second product
      id: "p-102",
      // Name title shown on the item card
      title: "Mechanical RGB Gaming Keyboard",
      // Unit price in USD for calculation and display
      price: 129.50,
      // Category used for dropdown filtering
      category: "Accessories",
      // Boolean indicator for product stock availability
      inStock: true,
      // Numerical user feedback rating out of 5.0
      rating: 4.6,
      // External image CDN URL for rendering card media
      imageUrl: "https://images.unsplash.com/photo-1587829741301-dc798b83add3?w=400&auto=format&fit=crop&q=80"
    // Closing boundary of second item definition
    },
    // Beginning definition of third item (p-103)
    {
      // Unique identifier assigned to third product
      id: "p-103",
      // Name title shown on the item card
      title: "Ergonomic Mesh Office Chair",
      // Unit price in USD for calculation and display
      price: 349.00,
      // Category used for dropdown filtering
      category: "Furniture",
      // Boolean indicator set to false (Out of Stock)
      inStock: false,
      // Numerical user feedback rating out of 5.0
      rating: 4.3,
      // External image CDN URL for rendering card media
      imageUrl: "https://images.unsplash.com/photo-1586023492125-27b2c045efd7?w=400&auto=format&fit=crop&q=80"
    // Closing boundary of third item definition
    },
    // Beginning definition of fourth item (p-104)
    {
      // Unique identifier assigned to fourth product
      id: "p-104",
      // Name title shown on the item card
      title: "4K Ultra-HD Monitor 27-inch",
      // Unit price in USD for calculation and display
      price: 419.99,
      // Category used for dropdown filtering
      category: "Electronics",
      // Boolean indicator for product stock availability
      inStock: true,
      // Numerical user feedback rating out of 5.0
      rating: 4.9,
      // External image CDN URL for rendering card media
      imageUrl: "https://images.unsplash.com/photo-1527443224154-c4a3942d3acf?w=400&auto=format&fit=crop&q=80"
    // Closing boundary of fourth item definition
    }
  // Closing boundary for products data array
  ];
```

2. Header Component (src/components/Header.jsx)

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-3 > react-user-product-catalog > src > components > Header.jsx > ...

```jsx
// Import the core React library to enable JSX syntax
import React from 'react';

// Define and export the functional Header component with destructured props
export default function Header({ user, cartItemCount, cartTotal }) {
  // Begin return statement containing the component UI layout
  return (
    // Top-level HTML header element styled with the navbar CSS class
    <header className="navbar">
      {/* Container wrapper for application branding */}
      <div className="brand">
        {/* Primary application header heading */}
        <h1>TechMarket</h1>
      {/* Close the brand wrapper container */}
      </div>

      {/* Wrapper section grouping cart status and user profile */}
      <div className="user-cart-section">
        {/* Badge container displaying shopping cart details */}
        <div className="cart-badge">
          {/* Span displaying cart item count and formatted total cost */}
          <span>🛒 Cart: <strong>{cartItemCount}</strong> items (${cartTotal.toFixed(2)})</span>
        {/* Close the cart badge container */}
        </div>
        {/* Container wrapper holding user avatar and profile details */}
        <div className="user-profile">
          {/* Profile image tag loading the user avatar */}
          <img src={user.avatar} alt={user.name} className="avatar" />
          {/* Container holding textual user details */}
          <div className="user-details">
            {/* Paragraph element displaying the full user name */}
            <p className="user-name">{user.name}</p>
            {/* Span element displaying the assigned user role */}
            <span className="user-role">{user.role}</span>
          {/* Close the user details container */}
          </div>
        {/* Close the user profile container */}
        </div>
      {/* Close the user-cart section container */}
      </div>
    {/* Close the main navigation header element */}
    </header>
  // End of return statement
  );
// End of Header component function
}
```

3. Filter Controls (src/components/FilterBar.jsx)

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-3 > react-user-product-catalog > src > components > FilterBar.jsx >

```jsx
import React from 'react';
export default function FilterBar({
  searchQuery,
  setSearchQuery,
  selectedCategory,
  setSelectedCategory,
  onlyInStock,
  setOnlyInStock,
  categories = []
}) {
  // Defensive check: Ensure categories is a valid array
  const categoryList = Array.isArray(categories) ? categories : [];
  return (
    <div className="filter-bar">
      {/* Search Input */}
      <input
        type="text"
        placeholder="Search products..."
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
        className="search-input"
      />
      {/* Category Dropdown */}
      <select
        value={selectedCategory}
        onChange={(e) => setSelectedCategory(e.target.value)}
        className="category-select"
      >
        {/* Default option to show all products */}
        <option value="All">All Categories</option>
        {/* Map over the array to generate individual <option> elements */}
        {categoryList.map((categoryName) => (
          <option key={categoryName} value={categoryName}>
            {categoryName}
          </option>
        ))}
      </select>
      {/* Stock Checkbox Filter */}
      <label className="checkbox-label">
        <input
          type="checkbox"
          checked={onlyInStock}
          onChange={(e) => setOnlyInStock(e.target.checked)}
        />
        In Stock Only
      </label>
    </div>
  );
}
```

4. Product Card Component (src/components/ProductCard.jsx)

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-3 > react-user-product-catalog > src > components > ProductCard.jsx >

```jsx
// Import the core React library
import React from 'react';

// Export default functional ProductCard component with destructured props
export default function ProductCard({ product, onAddToCart, cartQuantity }) {
  // Return the JSX markup structure
  return (
    // Main card wrapper div with dynamic CSS class for out-of-stock items
    <div className={`product-card ${!product.inStock ? 'out-of-stock' : ''}`}>
      {/* Wrapper div for image and category tag */}
      <div className="image-wrapper">
        {/* Render product thumbnail image */}
        <img src={product.imageUrl} alt={product.title} />
        {/* Render category tag badge */}
        <span className="category-tag">{product.category}</span>
      {/* End of image wrapper container */}
      </div>

      {/* Container div for product details */}
      <div className="card-details">
        {/* Render product title heading */}
        <h3>{product.title}</h3>
        {/* Render product rating out of 5 */}
        <p className="rating">⭐ {product.rating} / 5.0</p>
        {/* Render formatted product price */}
        <p className="price">${product.price.toFixed(2)}</p>

        {/* Action buttons wrapper div */}
        <div className="card-actions">
          {
            /* Check stock status and render button conditionally */
            product.inStock ? (
              /* Active add-to-cart button element */
              <button className="add-btn" onClick={() => onAddToCart(product)}>
                {/* Button label with conditional cart item count */}
                Add to Cart {cartQuantity > 0 && `(${cartQuantity})`}
              {/* End of add-to-cart button */}
              </button>
            ) : (
              /* Disabled button element for out-of-stock items */
              <button className="disabled-btn" disabled>
                {/* Out of stock label text */}
                Out of Stock
              {/* End of disabled button */}
              </button>
            )
            /* End of conditional ternary operation */
          }
        {/* End of card actions container */}
        </div>
      {/* End of card details container */}
      </div>
    {/* End of main product card container */}
    </div>
  // End of return statement
  );
// End of ProductCard component function
}
```

5. Product Grid Container (src/components/ProductGrid.jsx)

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-3 > react-user-product-catalog > src > components > ProductGrid.jsx >

```jsx
// Import the core React library
import React from 'react';
// Import the child ProductCard presentation component
import ProductCard from './ProductCard';

// Component that renders either an empty state message or a responsive grid of products
export default function ProductGrid({ products, onAddToCart, cart }) {
  // Check if no products match the current active filters
  if (products.length === 0) {
    return (
      <div className="empty-state">
        {/* Display feedback message when filter results are empty */}
        <p>No products match your current filters.</p>
      </div>
    );
  }

  // Render main product grid layout when matching items exist
  return (
    <div className="product-grid">
      {/* Map through each product in the filtered list */}
      {products.map((product) => {
        // Search cart array to check if the current product is already added
        const cartItem = cart.find(item => item.id === product.id);
        // Extract quantity added to cart, defaulting to 0 if not present
        const quantity = cartItem ? cartItem.quantity : 0;

        // Return ProductCard component populated with item details and cart state
        return (
          <ProductCard
            key={product.id}
            product={product}
            onAddToCart={onAddToCart}
            cartQuantity={quantity}
          />
        );
      })}
    </div>
  );
}
```

6. Orchestrator Component (src/App.jsx)

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-3 > react-user-product-catalog > src > App.jsx > ...

```jsx
// Import React library and the useState hook for managing state
import React, { useState } from 'react';
// Import mock user data and initial product catalog array
import { CURRENT_USER, INITIAL_PRODUCTS } from './data/mockData';
// Import top navigation header component
import Header from './components/Header';
// Import filter and search control component
import FilterBar from './components/FilterBar';
// Import product grid container component
import ProductGrid from './components/ProductGrid';
// Import global CSS styles
import './App.css';

// Main orchestrator component that manages application state and data flow
export default function App() {
  // Store full list of products in state
  const [products] = useState(INITIAL_PRODUCTS);
  // Store shopping cart items array in state
  const [cart, setCart] = useState([]);
  // Store search input filter string in state
  const [searchQuery, setSearchQuery] = useState('');
  // Store selected category filter string in state
  const [selectedCategory, setSelectedCategory] = useState('All');
  // Store in-stock checkbox toggle boolean in state
  const [onlyInStock, setOnlyInStock] = useState(false);

  // Dynamically generate unique category list from products array
  const categories = [...new Set(products.map((p) => p.category))];

  // Handler function to add new items or increment quantity in cart
  const handleAddToCart = (productToAdd) => {
    // Update cart using functional state updater to avoid stale closures
    setCart((prevCart) => {
      // Find if item already exists in cart array
      const existing = prevCart.find((item) => item.id === productToAdd.id);
      // Check if product is already in cart
      if (existing) {
        // Increment quantity of matched item by 1
        return prevCart.map((item) =>
          item.id === productToAdd.id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      }
      // Add new product object with initial quantity of 1
      return [...prevCart, { ...productToAdd, quantity: 1 }];
    });
  };

  // Derive filtered product list on every render based on active user controls
  const filteredProducts = products.filter((p) => {
    // Check if title includes search string (case-insensitive)
    const matchesSearch = p.title.toLowerCase().includes(searchQuery.toLowerCase());
    // Check if category matches dropdown or if 'All' is selected
    const matchesCategory = selectedCategory === 'All' || p.category === selectedCategory;
    // Check if item is in stock when stock filter is active
    const matchesStock = !onlyInStock || p.inStock;
    // Return boolean determining if item passes all three criteria
    return matchesSearch && matchesCategory && matchesStock;
  });

  // Calculate total count of all items combined in cart
  const totalCartItems = cart.reduce((sum, item) => sum + item.quantity, 0);
  // Calculate total monetary value of items in cart
  const totalCartCost = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);

```

```
67    // Render component UI tree
68    return (
69      /* Outer layout wrapper container */
70      <div className="app-container">
71        {/* Header component displaying user details and cart totals */}
72        <Header
73          user={CURRENT_USER}
74          cartItemCount={totalCartItems}
75          cartTotal={totalCartCost}
76        />
77
78        {/* Main content body container */}
79        <main className="content">
80          {/* Section title */}
81          <h2>Product Catalog</h2>
82
83          {/* Filter controls component for search, category, and stock filtering */}
84          <FilterBar
85            searchQuery={searchQuery}
86            setSearchQuery={setSearchQuery}
87            selectedCategory={selectedCategory}
88            setSelectedCategory={setSelectedCategory}
89            onlyInStock={onlyInStock}
90            setOnlyInStock={setOnlyInStock}
91            categories={categories}
92          />
93
94          {/* Product grid container displaying filtered products */}
95          <ProductGrid
96            products={filteredProducts}
97            onAddToCart={handleAddToCart}
98            cart={cart}
99          />
100       {/* Close main content section */}
101       </main>
102     {/* Close outer layout wrapper */}
103     </div>
104   );
105 }
```

7. Global Application Styles (src/App.css)

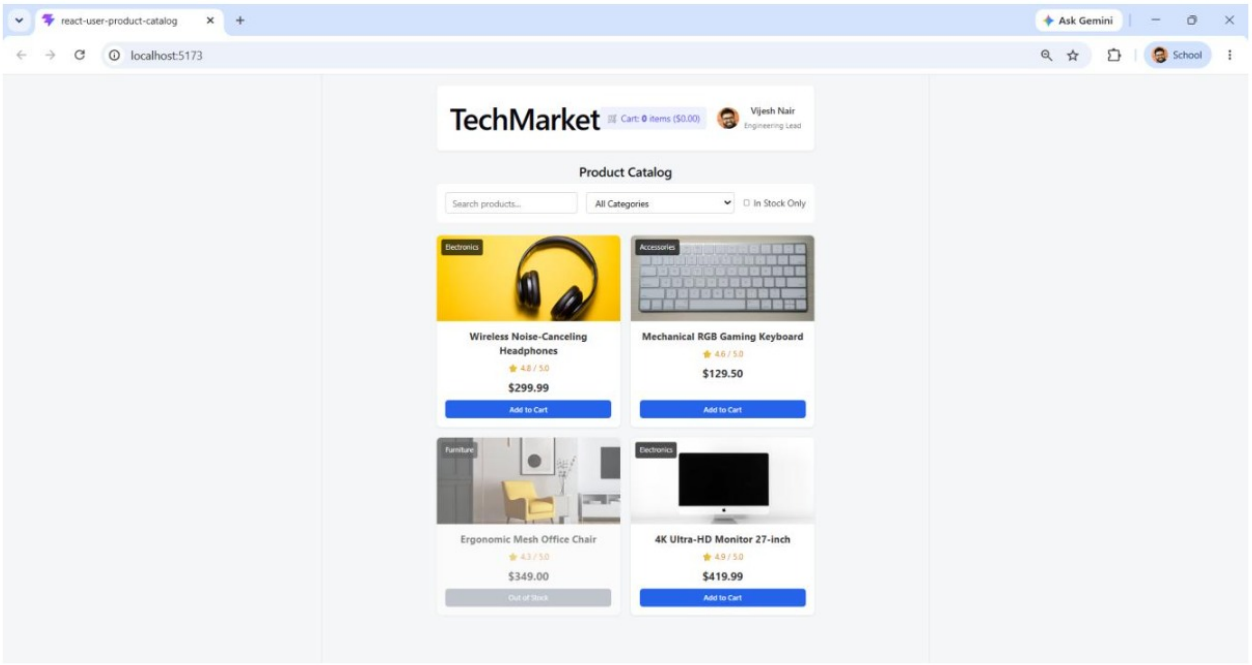
D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-3 > react-user-product-catalog > src > # App.css

```css
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: 'Inter', system-ui, -apple-system, sans-serif;
}
body {
  background-color: #f4f6f8;
  color: #333;
}
.app-container {
  max-width: 1100px;
  margin: 0 auto;
  padding: 20px;
}
/* Header */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: #ffffff;
  padding: 16px 24px;
  border-radius: 8px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
  margin-bottom: 24px;
}
.user-cart-section {
  display: flex;
  align-items: center;
  gap: 20px;
}
.cart-badge {
  background: #eef2ff;
  color: #4f46e5;
  padding: 8px 12px;
  border-radius: 6px;
  font-size: 0.9rem;
}
.user-profile {
  display: flex;
  align-items: center;
  gap: 10px;
}
.avatar {
  width: 40px;
  height: 40px;
  border-radius: 50%;
}
.user-name {
  font-weight: 600;
  font-size: 0.9rem;
}
.user-role {
  font-size: 0.75rem;
  color: #6b7280;
}
/* Filters */
.filter-bar {
  display: flex;
  gap: 16px;
  align-items: center;
  margin-bottom: 24px;
  background: #ffffff;
  padding: 16px;
  border-radius: 8px;
}
```

```
.search-input, .category-select {
  padding: 8px 12px;
  border: 1px solid #d1d5db;
  border-radius: 6px;
  outline: none;
  font-size: 0.9rem;
}
.search-input {
  flex-grow: 1;
}
.checkbox-label {
  display: flex;
  align-items: center;
  gap: 6px;
  font-size: 0.9rem;
  cursor: pointer;
}
/* Product Grid */
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));
  gap: 20px;
}
.product-card {
  background: #ffffff;
  border-radius: 8px;
  overflow: hidden;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
  display: flex;
  flex-direction: column;
}
.product-card.out-of-stock {
  opacity: 0.6;
}
.image-wrapper {
  position: relative;
  height: 160px;
}
.image-wrapper img {
  width: 100%;
  height: 100%;
  object-fit: cover;
}
.category-tag {
  position: absolute;
  top: 8px;
  left: 8px;
  background: rgba(0, 0, 0, 0.7);
  color: #fff;
  padding: 2px 8px;
  font-size: 0.7rem;
  border-radius: 4px;
}
.card-details {
  padding: 16px;
  display: flex;
  flex-direction: column;
  flex-grow: 1;
}
.card-details h3 {
  font-size: 1rem;
  margin-bottom: 6px;
}
.rating {
  font-size: 0.8rem;
  color: #d97706;
```

```
133     }
134     .price {
135       font-size: 1.1rem;
136       font-weight: bold;
137       margin: 10px 0;
138     }
139     .card-actions {
140       margin-top: auto;
141     }
142     .add-btn {
143       width: 100%;
144       padding: 8px;
145       background: #2563eb;
146       color: white;
147       border: none;
148       border-radius: 6px;
149       cursor: pointer;
150       font-weight: 500;
151     }
152     .add-btn:hover {
153       background: #1d4ed8;
154     }
155     .disabled-btn {
156       width: 100%;
157       padding: 8px;
158       background: #9ca3af;
159       color: white;
160       border: none;
161       border-radius: 6px;
162       cursor: not-allowed;
163     }
164     .empty-state {
165       text-align: center;
166       padding: 40px;
167       background: #fff;
168       border-radius: 8px;
169       color: #6b7280;
170     }
```

Output:



Conclusion: -

Thus, we have designed and implemented a simple React application, a Product Catalog dashboard using functional components and JSX.